TDS6213 Data Structures and Algorithms

Trimester October 2024

Bus Ticket Management System

Group 1

Lab Section: 2E/3E

Project Prepared by:

	Name	Student ID	Major
1	Megat Nur Hasanal Putera bin Norrasidi	1211112299	DCN
2	Muhammad Nasri bin Nasharuddin	1211112317	BIA
3	Azim Hakim bin Azizullail	1211100984	BIA
4	Muhammad Eusoff Aminurrashid Bin Shukri	1211112228	ST

FACULTY OF INFORMATION SCIENCE & TECHNOLOGY

MULTIMEDIA UNIVERSITY

MALAYSIA

15/12/2024

Contents

Introduction

The inefficiencies of conventional ticketing procedures at bus terminals are addressed by the Bus Ticket Management System, a complete solution. These issues, which together reduce passenger happiness and operational effectiveness, include lengthy lines, duplicate reservations, a lack of real-time seat availability, and a lack of operational transparency. The reputation of travel agencies and bus terminal management are badly impacted by manual operations, which frequently cause delays, resource waste, and operator and customer displeasure.

In order to address these problems, this research explores the creation of an automated system. Because the suggested system is based on strong data structures and algorithms, scalability, efficiency, and dependability are guaranteed. Real-time seat tracking, updated passenger counts, streamlined booking procedures, and automated refund systems are some of the main benefits. Additionally, the system incorporates innovative functionalities such as the Bus Points system to incentivize passengers for delays and offer discounts on future bookings.

The design and implementation of this system prioritize user convenience, operational efficiency, and customer satisfaction. By integrating modern software solutions with essential transport management requirements, the project seeks to revolutionize ticket management processes, improve the travel experience, and foster better resource utilization.

Problem Statements

The current ticket management system at the bus terminal, which focuses on physical counter sales, presents significant challenges for key stakeholders, including bus terminal management, travel companies, bus operators, and passengers. These challenges stem from inefficiencies in ticketing processes, lack of real-time information, and operational mismanagement.

Buses frequently experience delays arriving at the station on time, leading to passenger dissatisfaction and reputational damage for both the travel company and terminal management. **Passengers often encounter errors in ticketing and seat allocation**, such as double-booked seats, which disrupt their experience and cause conflicts with operators.

The inefficient ticketing system also makes it difficult for passengers to obtain tickets promptly, resulting in excessively long queues at counters. Compounding this issue is the **lack of real-time information about bus availability**, causing passengers to wait in line only to discover their desired bus is already fully booked.

Bus operators face difficulties tracking passenger counts accurately, increasing the risk of leaving passengers behind at terminals or intermediary stops during the trip. Additionally, **tickets often lack essential details such as platform numbers or bus identification**, making it harder for passengers to navigate their journey smoothly.

The current system also fails to address refund requests efficiently, discouraging terminals from offering refunds. As a result, passengers are left financially disadvantaged in emergencies when refunds are necessary.

These inefficiencies not only frustrate passengers but could also compromise their safety during emergencies or route disruptions. Additionally, the operational shortcomings result in wasted resources, such as fuel for delayed or partially empty buses, and missed opportunities for operators to maximize revenue and improve customer loyalty.

A robust, automated ticket management system is urgently needed to address these inefficiencies, ensure passenger satisfaction, and improve the overall operations of bus terminals and travel companies.

Objectives

The objectives of our proposed bus ticket management system are:

1. **Bus Points for as Compensation:** Since delays are issues which we cannot measure using existing methods, our system will provide 10 bonus points to passengers for every 10 minutes of delay.
2. **Lock Concurrency Control:** The system will reserve the desired seat during the booking process to prevent other passengers from accessing it simultaneously. The seat will remain unavailable until the passenger either cancels the booking or completes the payment, at which point the lock will be released.
3. **Simplified Booking Process**: Inefficiencies in the ticketing system are due to many factors which the program alone cannot solve. Thus, our system will only require the essential booking information to book a ticket.
4. **Real-Time Seat Tracking:** Shows the real-time information of available buses and their number of unreserved seats.
5. **Passenger Tracking:** The passenger count of each bus will be sent to the bus operators 10 minutes before departure.
6. **Provide Comprehensive Ticket Details:** Include detailed information on tickets such as bus ID, travel company, platform, seat, date and time of departure.
7. **Streamline Refund Management:** Enable an automated process to address refund requests within 24 hours, reducing manual intervention. A 20% cancellation fee will be subtracted from the original ticket price as a penalty to discourage refunds.

List the functions/features in the system

The functions and features of the Bus Ticket Management System are as follows:

Functions:

1. **Book new ticket** – Add a new ticket record
2. **Edit passenger information** – Edit a passenger record
3. **Cancel booking** – Delete the existing ticket record
4. **Show ticket information**– Displays information from ticket record
5. **Search for bus at a specific date** – Search for bus record
6. **Sort bus by prices** – Sort bus records

Features:

1. **Seat Reservation System**: Allows passengers to select and reserve seats in real-time with concurrency control.
2. **Real-time Bus Availability:** Displays the availability of seats and buses for selected routes.
3. **Simplified Booking:** Book only with the essential information which are: name, phone number, bus and seat number.
4. **Comprehensive Ticketing:** Provide detailed information on tickets such as the ticket ID, passenger's name, passenger's phone number, bus ID, travel company, platform, seat, date and time of departure.
5. **Passenger Count Tracker:** Counts and displays the number of passengers for the bus operators.
6. **Refund and Cancellation System:** Automated refund and cancellation system with only 20% cancellation fee. Refunds can only be requested up to 15 hours before departure.
7. **Purchase History:** Displays the ticket purchase history of the passenger.
8. **Bus Points System:** Provide 1 Bus Point per RM based on the price of the ticket purchased and provide 10 Bus Points for every 10 minutes of delay in departure. Bus Points can be redeemed as a discount.

ADT Specification (at least 2 pages)

Passenger ADT

Represents each passenger/user of the system.

Data Items:

- name: string
- phoneNumber: string
- busPoints: integer
- ticketList: TicketList*

Operations:

1. **Passenger()**

Results:

- Default constructor. Create a passenger instance with empty values.

2. **Passenger(string name, string phoneNumber)**

Results:

- Constructor. Create a passenger class with the given data.
- Initialize busPoints to zero and an empty linked list for ticketList

3. **~Passenger()**

Results:

- Destructor. Destroys the instance of Passenger.
- ticketList pointer becomes nullptr.

4. **bool redeemPoints(int points)**

Requirements:

- points must not exceed current busPoints or less than zero

Results:

- Subtract points from busPoints.
- Return true if successful, false if unsuccessful.

5. **bool earnPoints(int points)**

Results:

- Add points to busPoints.
- Return true if successful, false if unsuccessful.

6. **bool editInfo()**

Results:

- Edits the data of passenger class such as name and phoneNumber.
- Return true if successful

7. **void display() const**

Results:

- Displays all the information of Passenger object.

Getter methods:

8. **string getName() const**
9. **string getPhoneNumber() const**
10. **int getBusPoints() const**
11. **TicketList* getTicketList() const**

TicketList ADT

The linked list for Tickets of a passenger.

Data Items:

- TicketNode: struct – contains item: ItemType* that points to Ticket item and next: TicketNode* that points to the next item
- count: integer
- headPtr: TicketNode*

Operations:

1. **TicketList()**

Results:

- Constructor. Create an instance of list TicketList.
- Initialize the count as zero and headPtr as nullptr.

2. **~TicketList()**

Results:

- Destructor. Destroy the instance of TicketList.
- Delete all instances of TicketNode.

3. **bool isEmpty() const**

Results:

- Returns true if list is empty, and false if list is not.

4. **int getCount() const**

Results:

- Return the number of tickets previously booked.

5. **bool addTicket(ItemType* item)**

Results:

- Add a pointer to Ticket to the linked list.
- Returns true if successful, false if unsuccessful.

6. **bool removeTicket(int index)**

Requirements:

- list must not be empty
- index must be within 1 to the number of items

Results:

- Removes ticket at an index from the list.
- Returns true if successful, false if unsuccessful.

7. **ItemType* getTicket(int index)**

Requirements:

- index must be within 1 to the number of items

Results:

- Returns a pointer to Ticket from the linked list at an index

---

Ticket ADT

Represents the ticket bought by the passenger.

Data Items:

- ticketCount: int
- ticketID: int
- passengerName: string
- passengerPhone: string
- busID: integer
- busRoute: string
- busOperator: string
- busCompany: string
- seatNumber: integer
- platformNumber: int
- price: float
- <<const>> statusType: string[]
- statusIndex: integer

Operations:

1. **Ticket()**

Results:

- Default constructor. Create an instance of Ticket with blank information.

2. **Ticket(string passengerName, string passengerPhone, int busID, string busRoute, string busOperator, string busCompany, int seatNumber, int platformNumber, float price)**

Results:

- Constructor. Create an instance of Ticket with the given data.
- Initialize the status as "Booked".
- Initialize ticketID as ticketCount and increase ticketCount by 1.

3. **~Ticket()**

Results:

- Destructor. Destroys the instance of Ticket.

4. **bool updateStatus(int status)**

Requirements:

- status must be either 0 or 1.

Results:

- Updates the status of the ticket.
- Returns true if success, or false if fail due to invalid input.

5. **void display() const**

Results:

- Displays all information of Ticket object

Getter methods:

6. **int getTicketID() const**
7. **string getPassengerName() const**
8. **string getPassengerPhone() const**
9. **int getBusID() const**
10. **string getBusRoute() const**
11. **string getBusCompany() const**
12. **int getSeatNumber() const**
13. **int getPlatform() const**
14. **float getPrice() const**
15. **string getStatus() const**

Bus ADT

Represents the bus.

Data Items:

- busCount: integer
- <<const>> numberOfSeats: integer
- busID: integer
- route: string
- busCompany: string
- operatorName: string
- platformNumber: int
- departureTime: string
- price: float
- seats: Boolean[numberOfSeats]
- passengers: Passenger*[numberOfSeats]

- passengerCount: integer

Operations:

1. **Bus()**

Results:

- Default constructor. Create an instance of Bus with empty values.
- Initialize seats as an array of Boolean true values, passengerCount to zero.
- Initialize the busID equal to busCount and increase busCount by 1.

2. **Bus(string route, string busCompany, string operatorName, int platformNumber, string departureTime, float price)**

Results:

- Constructor. Create an instance of Bus with the following data.
- Initialize seats as an array of Boolean true values, passengerCount to zero.
- Initialize the busID equal to busCount and increase busCount by 1.

3. **~Bus()**

Results:

- Destructor. Destroys the instance of Bus.
- Sets all Passenger pointer in array passengers[] to nullptr

4. **bool lockSeat(int seatNumber)**

Requirements:

- seatNumber must be more than zero and less than or equal to numberOfSeats.
- The seatNumber must not be locked already.

Results:

- Mark a seat as locked.
- Return true if successful, false if unsuccessful.

5. **bool releaseSeat(int seatNumber)**

Requirements:

- seatNumber must be more than zero and less than or equal to numberOfSeats.
- The seatNumber must be locked.

Results:

- Releases a seat from being locked

- Return true if successful, false if unsuccessful.

6. **bool isAvailable() const**

Results:

- Check if all the seats on the bus are taken.
- Return true if not, false if yes.

7. **bool addPassenger(Passenger* passenger, int seatNumber)**

Requirements:

- The passengerCount is still less than numberOfSeats
- seatNumber is a valid number between 1 to numberOfSeats
- The seat cannot already be booked
- passenger must not be a null pointer

Results:

- Adds a pointer to Passenger to the passengers array at index seatNumber
- Add passengerCount by 1
- Returns true if successful, false if unsuccessful

8. **bool removePassenger(int seatNumber)**

Requirements:

- passengerCount must be more than zero
- seatNumber must be a valid number between 1 to numberOfSeats
- There is already a passenger at seatNumber

Results:

- Delete the pointer to Passenger at index seatNumber
- Subtract passengerCount by 1
- Returns true if successful, false if unsuccessful

9. **void display() const**

Results:

- Prints all information about the bus

Getter methods

10. **int getBusID() const**

11. **string getRoute() const**
12. **string getOperator() const**
13. **int getPlatformNumber() const**
14. **int getEmptySeats() const**
15. **int getPassengerCount() const**
16. **Passenger* getPassenger(int seatNumber)**

Requirements:

- seatNumber must be a valid number between 1 to numberOfSeats

Results:

- Returns the pointer to Passenger at index seatNumber

BookingManager

Manages all ticket bookings.

Data Items:

- refundManager: RefundManager
- pointsManager: PointsManager

Operations:

1. **BookingManager()**

Results:

- Default constructor.

2. **~BookingManger()**

Results:

- Default destructor

3. **bool bookTicket(Passenger* passenger, Bus* bus, int seatNumber)**

Requirements:

- The seat must be not be locked
- seatNumber must be valid

Results:

- Book ticket for passenger at bus and seatNumber
- Adds passenger to the list of passengers in the bus
- Pay for the ticket
- Apply discount using bus points

- Add bus points according to ticket price
- Create new ticket and assign it to the TicketList of the passenger

4. **bool cancelTicket(Passenger* passenger, Bus* bus, int ticketID, int hoursBeforeDeparture)**

Requirements;

- the ticketID must be valid

Results:

- Passenger will be removed from the bus
- Lock on the seat will be released
- Call RefundManager to handle refunding
- Cancel the ticket

5. **bool showTickets(Passenger* passenger)**

Results:

- Displays all the tickets of a passenger

6. **bool searchBusByID(Bus* buses[], int busCount, int busID)**

Results:

- Find and display a bus by its busID

7. **Bus* getBusByID(Bus* buses[], int busCount, int busID)**

Results:

- Find and return a bus by its busID

8. **bool sortBusByPrice(Bus* buses[], int busCount)**

Results:

- Sort the array of buses by price ascending

9. **bool showBuses(Bus* buses[], int busCount)**

Results:

- Displays all the buses available

10. **bool printTicket(Passenger* passenger)**

Results:

- Print tickets of a passenger to a file.txt

RefundManager

Manages all refunds.

Data Items:

- <<const>> refundPercent: float
- pointsManager: PointsManager

Operations:

1. **RefundManager()**

Results:

- Default constructor.

2. **~RefundManager()**

Results:

- Default destructor

3. **bool processRefund(Ticket* ticket, Passenger* passenger, int hoursBeforeDeparture)**

Requirements:

- hoursBeforeDeparture must be more than 15 hours

Results:

- Calculate refund amount based on the refundPercent of the class.
- Pay back the refund to passenger
- Update the status of the ticket to Cancelled
- Subtract bus points from the user.

PointsManager

Handles all bus points related operations.

Data Items:

- <<const>> pointsPerRM: integer

Operations:

1. **PointsManager()**

Results:

- Default constructor

2. **~PointsManager()**

Results:

- Default destructor.

3. **bool givePoints(Passenger* passenger, int points)**

Results:

- Add points to passenger's total bus points

4. **bool deductPoints(Passener* passenger, int points)**

Requirements:

- The points must be more than zero and less than the passenger's bus points

Results:

- Points are deducted from passenger's total bus points

5. **float applyDiscount(Passenger* passenger, float price)**

Requirements:

- Must use at least 100 points to get discount.

Results:

- Calculate the bus points used and deduct it from the ticket price
- Deduct points from passengers

QueueManager

Handles passenger queue and its operations.

Data item:

- QueueNode: struct – contains item: QueueItemType* and next: QueueNode*
- backPtr: QueueNode*
- frontPtr: QueueNode*

Operations:

1. **QueueManager()**

Results:

- The default constructor

2. **~QueueManager()**

Results:

- The default destructor

3. **bool enqueue(QueueItemType* newItem)**

Results:

- Adds a new item to the back of the queue
- If its empty, frontPtr will be the same as backPtr after insertion

4. **bool dequeue()**

Requirements:

- Queue must not be empty

Results:

- Remove the front most item in the queue

5. **bool isEmpty() const**

Results:

- Return true if the queue is empty

6. **bool addPassenger()**

Results:

- Add passenger to the queue by calling the enqueue method

7. **bool removePassenger()**

Results:

- Removes passenger from queue by calling the dequeue method

8. **bool showPassengers()**

Requirements:

- The queue must not be empty

Results:

- Displays all the passengers in the queue

9. **QueueItemType* getFront()**

Requirements:

- Queue cannot be empty

Results:

- Return the front most item in the queue

10. void displayMenu()

Results:

- Displays the menu for the Queue Manager
- Have option to add passenger to queue, remove, show and exit the Queue Manager.

Implementation Details

Data Structures:

1. **Linked List:**

The linked list is chosen for its dynamic and flexible nature, making it a better alternative to arrays in this context. It allows the TicketList to accommodate any number of tickets seamlessly without requiring a predefined size. This adaptability is crucial for handling varying ticket volumes efficiently.

2. **Queue:**

A queue is implemented to simulate real-life passenger queues, ensuring that passengers are served in a fair and orderly manner. This data structure adheres to the first-come, first-serve principle, where the passenger at the front of the queue is always served first. The virtual queue system reflects real-world operations, making the solution intuitive and user-friendly.

Sorting:

1. **Bubble Sort:**

Bubble sort is employed for its simplicity and minimal memory requirements, making it suitable for the current system. Given that the bus array being sorted typically contains a small number of elements, the performance trade-offs of bubble sort are negligible. Its straightforward implementation also makes it an efficient choice for this scenario.

Searching:

1. **Linear Search:**

Linear search is used for traversing the linked list due to its compatibility with the sequential nature of this data structure. Since a linked list requires visiting each node one by one, linear search aligns naturally with this approach. While other search algorithms like binary search might offer faster performance in sorted arrays, linear search is more appropriate for navigating unsorted linked lists effectively.

Task Distribution Table

Programming :

Name	ID	Tasks
Megat Nur Hasanal Putera bin Norrasidi	1211112299	Code BookingManager, PointsManager and main function. Test and troubleshoot program.
Muhammad Nasri bin Nasharuddin	1211112317	Code Passenger ADT and RefundManager Test and troubleshoot program
Azim Hakim bin Azizullail	1211100984	Code Ticket ADT and TicketList ADT. Test and troubleshoot program
Muhammad Eusoff Aminurrashid Bin Shukri	1211112228	Code Bus ADT, QueueManager. Test and troubleshoot program

Report :

Name	ID	Tasks
Megat Nur Hasanal Putera bin Norrasidi	1211112299	Introduction, problem statements and objectives of the project.
Muhammad Nasri bin Nasharuddin	1211112317	Document list of functions and the ADT specifications.
Azim Hakim bin Azizullail	1211100984	The explanations and justifications of why we use the algorithms
Muhammad Eusoff Aminurrashid Bin Shukri	1211112228	Testing results and conclusion of the report.

Presentation :

Name	ID	Tasks
Megat Nur Hasanal Putera bin Norrasidi	1211112299	Do the introduction, problem statements and objectives of the project.
Muhammad Nasri bin Nasharuddin	1211112317	Explain the list of functions of the program and the ADT specifications used.
Azim Hakim bin Azizullail	1211100984	Explain the sample code.
Muhammad Eusoff Aminurrashid Bin Shukri	1211112228	Show demonstration and results of the project. Conclude the presentation.

Gantt Chart

PROJECT NAME	PROJECT DURATION	PROJECT START DATE	PROJECT END DATE
Bus Ticket Management System	12 weeks	4-Nov	22-Jan

Task ID	Task Description	Task Duration	Start Date	End Date	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10	Week 11	Week 12
1	Group Formation	1 week	4-Nov	10-Nov												
2	Project Topic Finalization	2 weeks	11-Nov	24-Nov												
3	Initial Project Planning	3 weeks	25-Nov	14-Dec												
4	Development Phase	4 weeks	15-Dec	12-Jan												
5	Extra Features Implementation	3 Days	13-Jan	15-Jan												
6	Final Submission Preparation	4 Days	16-Jan	19-Jan												
7	Project Presentation	3 Days	20-Jan	22-Jan												

Conclusion

The Bus Ticket Management System successfully resolves the urgent issues with manual ticketing methods. The technology guarantees a smooth experience for both passengers and operators by giving strict and systematic procedures and integrating intuitive features. Through real-time updates, concurrency management, and faster operations, it addresses typical problems including multiple bookings, lengthy lines, and ineffective refund procedures.

By providing discounts for delays, the implementation of a Bus Points system encourages passengers and increases consumer engagement and loyalty. An effective booking process, real-time seat availability, and comprehensive ticket information enable consumers to make well-informed decisions while reducing errors and conflicts. Additionally, passengers' trust is increased, and manual intervention is greatly reduced by the system's automated refund procedure, which resolves claims within 24 hours.

All in all, this project shows how technology may revolutionise transport systems. It offers a structure that is scalable and adaptable for future improvements, such as extension into larger geographic areas and integration with other modes of transportation. The Bus Ticket Management System is a major advancement in the management of public transportation since it increases passenger happiness, lowers expenses, and improves efficiency.